

This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

I want to compare different optimizers: GD, SignGD, Adam, Muon. What are their invariants/sensitivities? What are the reason we would use one over the other?

First explain the basic mechanics of each (assuming I already have basic understanding of each)

Great — since you already have the basics, let's treat this like a study session where we refine and deepen your understanding rather than restating what you know.

Before I dive in: **what level of depth are you aiming for?**

For example:

- high-level conceptual comparison,
- mathematical invariants (scale invariance, coordinate-wise invariance, etc.),
- geometric interpretation (e.g., induced norm, conditioning sensitivity),
- or practical “what breaks / what works well” guidance?

Give me a sense of which mix you want, and then we'll build up the comparison step by step.

mathematical invariants (scale invariance, coordinate-wise invariance, etc.),

geometric interpretation (e.g., induced norm, conditioning sensitivity),

Perfect — that's a great direction. Before we go deep, I want to get a quick read on where you are so I don't start at the wrong level.

Here's a **single quick question** for calibration:

When you think about an optimizer being *scale-invariant*, what is your current mental model?

For example, do you think of it in terms of:

probing question hinting at the fact that each of these affects scale of optimizer update

- invariance to **scaling the parameters**,
- invariance to **scaling the gradients**,
- or invariance to **re-parameterizing the model** (e.g., multiplying a layer by c and dividing the next by $1/c$)?

Give me your sense of that, and then I'll tune the explanations of GD, SignGD, Adam, and Muon around those invariants and their geometric views.

all of them? for linear layers we can scale the parameters or the gradients or reparameterize the model and they could be equivalent with the right scaling?

Great — that already tells me you're thinking in the right direction: *different optimizers treat those scalings very differently*, and that's exactly what creates their characteristic behavior.

Let's take this slowly and build the comparison around your point.

To keep this interactive, let me start with **one focused step**:

Suppose you take a linear layer $y = Wx$, and you replace W by cW .

If you don't change anything else, the network changes — but if you scale another connected layer appropriately, you can keep the *function* the same.

Here's the question for you:

When you scale $W \rightarrow cW$, what happens to the gradient $\nabla_W L$?

(Think purely mechanically: how does gradient scale with respect to parameter scaling in linear layers?)

Take a shot — your answer will determine how we line up the invariance properties of GD, SignGD, Adam, and Muon next.

scales gradient by c

Good — with that in mind, we can now line up how the optimizers react to that scaling. I'll walk you through this piece by piece, and I'll ask you to reason at each step so you internalize the geometric picture.

Step 1

You said: scaling $W \rightarrow cW$ makes the gradient scale $\nabla_W L \rightarrow c\nabla_W L$.

Now here's the key guiding question:

If an optimizer updates via $\Delta W = -\eta g$, what happens to the *relative effect* of the update when W is scaled by c ?

Think about whether the parameter update “feels bigger or smaller” compared to the parameter value after scaling.

Concretely: consider the ratio

$$\frac{\|\Delta W\|}{\|W\|}$$

How does this ratio change when you scale $W \rightarrow cW$?

Give me your reasoning on that ratio. This will directly lead into why GD is **not** scale-invariant but SignGD **is**, and how Adam and Muon land in between depending on their normalization structure.

the ratio won't change because they'll cancel out

I was not entirely sure why this relative change was important

Thought

Thought

Nice — your last step (ratio cancels) was exactly right, and it's a good launching pad. Below I'll give a compact, math-focused comparison for each optimizer along the axes you asked for: **invariants / sensitivities**, **geometric interpretation**, and **practical reasons to pick it**. I'll keep each block short and end with a single guiding question so we keep this interactive.

Quick notation

Let $g_t = \nabla_{\theta} L(\theta_t)$.

We'll treat parameters either as vectors (GD, SignGD, Adam) or as per-layer matrices W (Muon). $\|\cdot\|_2$, $\|\cdot\|_1$, $\|\cdot\|_{\infty}$ are the usual norms.

1) Gradient descent / SGD (vanilla)

Update: $\Delta\theta_t = -\eta g_t$ (or with momentum: $-\eta m_t$, $m_{t+1} = \beta m_t + (1 - \beta) g_t$).

Invariants / sensitivities

- **Gradient scaling:** If $g_t \mapsto c g_t$ then $\Delta \mapsto c \Delta$. If you *also* scale $\theta \mapsto c \theta$, the ratio $\|\Delta\| / \|\theta\|$ is unchanged (the cancellation you observed).
- **Coordinate-wise rescaling:** *Not invariant*. If you rescale individual coordinates (multiply coordinate i by α_i), the update interacts poorly — directions with small scale move slowly and large-scale coordinates move fast.
- **Rotations / basis change:** Not invariant to arbitrary rotations of parameter coordinates; the Euclidean geometry is tied to the current coordinate system.
- **Reparameterization (layer-scale swap):** If you multiply a layer by c and divide the next layer by $1/c$, the gradient magnitudes change nontrivially and GD will behave differently — so GD is sensitive to this kind of parameterization.

Geometric view

- GD is *steepest descent under the Euclidean norm* (for the unconstrained step of fixed step-size). That is, it follows the direction of fastest decrease per unit ℓ_2 -distance.
- Ill-conditioned Hessians (large condition number) $\rightarrow$ slow convergence along small-curvature directions (plateaus / narrow ravines).

When to use

- Simple baselines, small models, cases where preconditioning is unnecessary or you can tune learning rate + momentum.

- When you want behavior tied to true Euclidean geometry (e.g., analyses that assume isotropic noise).

2) SignGD / signSGD

Update: $\Delta\theta_t = -\eta \text{sign}(m_t)$ with optional momentum $m_{t+1} = \beta m_t + (1 - \beta) g_t$.

(See signSGD literature.)

$$\frac{\text{sign}(g) + 1}{2}$$

It was able to connect to each of the norms, 2, 1, infinity norm

Invariants / sensitivities

- **Gradient magnitude scaling:** *Invariant* to multiplicative scaling of the gradient: if $g \mapsto cg$ with $c > 0$, $\text{sign}(g)$ is unchanged (magnitude info lost).
- **Coordinate-wise rescaling:** If you rescale coordinate i by α_i , *sign* will flip only if α_i is negative — but magnitude changes are discarded, so signGD can be **robust** to heterogeneous coordinate magnitudes (but this is nuanced). The algorithm uses only direction (per-coordinate sign), so it is **coordinate-wise**: it treats coordinates independently.
- **Rotations:** Not invariant — a rotation mixes coordinates and the per-coordinate sign is not preserved.

Geometric view

- SignGD is (informally) the steepest descent under an ℓ_∞ -constraint on the step: if you restrict the step to have bounded ℓ_∞ -norm, the sign step is optimal direction-wise. It discards magnitude information and uses only direction (one-bit quantization per coordinate).
- This makes it robust to very noisy gradients and attractive for communication/compression in distributed setups.

Tradeoffs / when to use

- Pros: very robust to gradient scale and certain noise models; excellent for communication-constrained/distributed settings; surprisingly competitive with proper momentum/error-feedback.
- Cons: throws away magnitude information — can slow convergence in problems where magnitude encodes curvature (ill-conditioned directions). Use when gradient noise or compression is the bottleneck.

$$\frac{\text{sign}(g) + 1}{2}$$

3) Adam (coordinate-wise adaptive moment methods)

Update (sketch):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\Delta\theta_t = -\eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

(Kingma & Ba, 2015.)

$$\frac{\text{sign}(g) + 1}{2}$$

Invariants / sensitivities

- **Global scaling of gradients:** If $g \mapsto cg$ (global scalar), then $m \mapsto cm$ and $v \mapsto c^2v$ so $\hat{m} / (\sqrt{\hat{v}}) \approx$ sign-preserving and the factor c cancels — *Adam is invariant to global gradient scaling* (approximately, ignoring bias-correction constants).
- **Coordinate-wise rescaling:** Adam is *invariant to diagonal rescalings of coordinates* because v tracks per-coordinate second moments and rescales each coordinate by $\sqrt{v_i}$. So if one coordinate is measured in different units, Adam rescales updates to compensate.
- **Rotations / mixing:** *Not invariant* — Adam is diagonal (coordinate-wise) preconditioning only; it cannot capture cross-coordinate curvature like a full-matrix preconditioner would.
- **Reparameterization (layer-scale swaps):** Adam is less sensitive than plain SGD in many cases because per-parameter normalization compensates for scale differences, but it still depends on coordinate axes.

Geometric view

- Adam approximates a *diagonal* preconditioner: it rescales each coordinate by an estimate of recent root-mean-square gradient. Interpreted as approximate steepest descent under a diagonal Mahalanobis norm (metric $\text{diag}(\sqrt{v_t})$).
- Works well when curvature is approximately separable (or when per-coordinate statistics capture the important scale differences).

Tradeoffs / when to use

- Pros: fast initial progress, robust to learning-rate tuning, excellent for sparse gradients and heterogeneous coordinate scales.
- Cons: can generalize worse in some cases vs SGD with momentum; can be sensitive to hyperparameters (though AdamW + weight decay fixes some issues). Use as default for many tasks; switch to SGD+momentum for best generalization in vision tasks sometimes.

4) Muon (matrix / layer-aware optimizer)

Short description: Muon is a newer optimizer designed specifically for *matrix-shaped* parameters (hidden linear layers). It applies a per-matrix orthogonalization / normalization step (Newton–Schulz style iterations for matrix inverse-square-root-type operations), combined with momentum and carefully chosen per-parameter scaling. Empirical work shows it can improve conditioning and training speed for large LLMs and hidden layers; practical implementations pair Muon for hidden weights with AdamW for embeddings/heads.

Its summary of Muon was the most fuzzy/unclear, which makes not surprising since it is such a new optimizer

Update (informal):

- Compute a matrix-level transform of recent gradients or weight covariances, orthogonalize or normalize across rows/columns, then apply momentum and a rescaled update. (Implementations differ; the canonical description uses Newton–Schulz iterations to compute matrix inverse/square-root approximations.)

Invariants / sensitivities

- **Gradient scaling:** Muon's per-matrix normalization tends to cancel uniform gradient scaling, so like Adam it is robust to global scale changes.

- **Coordinate-wise rescaling:** Muon is *not* coordinate-diagonal — it works across rows/columns, so it can handle cross-coordinate couplings that Adam misses. This gives it invariance to certain reparameterizations that preserve matrix geometry (e.g., orthonormal transforms) better than coordinate-wise methods.
- **Rotations / basis changes on the matrix:** Because Muon works with matrix-level spectral information (singular values / covariances), it is **sensitive to and exploits** rotational structure — in fact, one benefit is reduced condition number (spectral preconditioning).

diagonal
V +1

Geometric view

- Muon behaves like a *matrix-aware preconditioner* that implicitly enforces spectral-norm-type constraints on weight matrices (it reduces condition numbers by orthogonalizing/normalizing updates). Several analyses place Muon in families related to natural-gradient or constrained steepest descent on matrix manifolds (e.g., Stiefel manifold / spectral-norm constraints). This gives it a geometry closer to a *low-rank / spectral-aware* metric rather than Adam's diagonal metric.

Stiefel
Bernstein

Tradeoffs / when to use

- Pros: can dramatically improve data-efficiency for large models (especially dense hidden layers), reduce ill-conditioning, and outperform AdamW in certain LLM training regimes. Works well with large batches and modern LLM recipes.
- Cons: extra matrix computations (Newton–Schulz iterations, orthogonalization) add overhead and communication complexity in distributed settings; can require pairing (e.g., Muon for hidden matrices + AdamW for embeddings/heads). Implementation details and LR-schedules matter; there are follow-up papers and variants improving scalability.

diagonal
V +1

Newton–Schulz
Documentation +1

Comparative table (short)

- **GD:** Euclidean steepest descent; sensitive to conditioning; cheap; baseline.
- **SignGD:** ℓ_∞ -style step; invariant to gradient magnitude; robust to noise/compression; loses curvature info.
- **Adam:** Diagonal adaptive preconditioning; invariant to per-coordinate scale; good for sparse/noisy gradients; misses cross-coordinate curvature.
- **Muon:** Matrix-level orthogonalization / spectral preconditioning; reduces condition numbers across rows/columns; leverages matrix geometry; higher per-step cost and engineering complexity.

diagonal
V

Stiefel
V

Newton–Schulz
Jordan +1

Small conceptual check (one question)

Which of these two invariances would you like to inspect next with a short derivation or a tiny example?

A. **Coordinate-wise (diagonal) rescaling invariance** — show algebraically why Adam cancels per-coordinate scalings but SGD does not.

B. **Rotation / basis-change sensitivity** — show with a 2D quadratic how a diagonal

preconditioner (Adam-like) can fail when Hessian eigenvectors are off-axis, and why a matrix-aware step like Muon could help.

Pick A or B and I'll work the math with you step-by-step.

 Sources

B then test my understanding

Great — let's do **B: rotation / basis-change sensitivity**.

I'll walk you through a clean 2D quadratic example, show why **diagonal** preconditioners (SGD, Adam-type) fail when curvature is *rotated*, and why **matrix-aware** preconditioners (Muon-type) fix it.

After that, I'll give you a test question that forces you to reason geometrically.

1. Set up a 2D quadratic in a *rotated* basis

Consider the loss

$$f(x) = \frac{1}{2} x^\top H x$$

It was able to give a good concrete example to test my understanding of Adam vs Muon

with Hessian

$$H = R(\theta) \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} R(\theta)^\top, \quad 0 < \lambda_1 \ll \lambda_2.$$

Where $R(\theta)$ is the 2D rotation matrix:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}.$$

So the quadratic bowl is elongated along the eigenvector associated with λ_1 , but that eigenvector is *not* aligned with coordinate axes.

Thus:

- Curvature directions are at angle θ .
- Coordinate axes are at angle 0 .
- Optimizers that treat coordinates independently (like SGD, Adam) are mismatched against the true curvature directions.

2. Gradient and steepest descent direction

For this quadratic:

$$\nabla f(x) = Hx.$$

The *optimal* direction of scaled movement (natural gradient / full Newton) is:

$$H^{-1} \nabla f(x) = x.$$

Meaning: the full curvature-aware step eliminates the rotation exactly — because H^{-1} cancels both the eigenvalues *and* the eigenvectors (rotations).

3. Why diagonal preconditioners fail under rotation

A diagonal preconditioner has the form:

$$P = \begin{pmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{pmatrix}.$$

The update is:

$$\Delta x = -P \nabla f(x) = -PHx.$$

But $PH = PR\Lambda R^\top$.

Because P is diagonal in the *coordinate* basis, and the eigenvectors of H are *rotated*, the diagonal P cannot undo the rotation R .

Consequence:

- Even if P perfectly matches eigenvalues ($\alpha_i \propto 1/\lambda_i$),
- the gradient is still rotated by R ,
- and diagonal scaling cannot un-rotate it.

This leads to the classic zig-zag behavior in rotated narrow valleys.

4. Adam suffers the same limitation

Adam's implicit preconditioner is:

$$P_t = \text{diag} \left(\frac{1}{\sqrt{v_t}} \right),$$

still diagonal.

Even though v_t adapts to per-coordinate curvature, it **cannot detect curvature lying in mixed directions** (eigenvectors not aligned with axes).

So:

- If the true Hessian eigenvectors are rotated,
- and the gradient direction changes along that rotation,
- Adam keeps updating in axis-aligned fashion → suboptimal zig-zag.

This is the reason Adam converges slowly on rotated quadratics unless the rotation is small.

5. Why a matrix-aware preconditioner (Muon-type) fixes this

If the optimizer constructs a **matrix** preconditioner, e.g.:

$$P \approx H^{-1/2} \text{ or anything that captures row/column covariance,}$$

then:

$$PH = H^{-1/2}H = H^{1/2} \approx \text{almost isotropic operator.}$$

This cancels the rotation R :

$$H^{1/2} = R \begin{pmatrix} \sqrt{\lambda_1} & 0 \\ 0 & \sqrt{\lambda_2} \end{pmatrix} R^\top.$$

Thus a matrix-aware step:

- de-rotates the gradient direction (aligns update with eigenvectors),
- equalizes curvature across eigen-directions,
- drives you straight to the minimum with no zig-zag,
- even when eigenvectors are arbitrarily rotated.

Muon does not compute $H^{-1/2}$ explicitly, but by normalizing/orthogonalizing weight matrices and using Newton–Schulz–style matrix iterations, it approximates matrix-level whitening and curvature correction that diagonal methods cannot do.

That’s the geometric heart of why Muon can outperform Adam in deep matrix-shaped layers.

6. Your comprehension test

Consider the Hessian

$$H = R(45^\circ) \begin{pmatrix} 1 & 0 \\ 0 & 100 \end{pmatrix} R(45^\circ)^\top.$$

You run Adam with a perfectly tuned diagonal preconditioner:

$$P = \begin{pmatrix} 1 & 0 \\ 0 & 0.01 \end{pmatrix}.$$

Question:

Suppose your current position is $x = (1, 0)$.

- What is the *true Newton step direction* (up to scaling)?
- What is the *Adam-style diagonal-preconditioned step direction* PHx ?
- Describe, in geometric terms, the exact way in which Adam’s step differs from Newton’s step for this rotated Hessian.

Try to compute the vectors explicitly. The computation is short but revealing.

Newton is $(-1, 0)$; Adam step is proportional to $(-1, 0.0098)$

Thought for 16s

Nice — your numbers are right. Quick verification and a short geometric comment, then a one-line test.

Verification (compact math).

With $R(45^\circ)$ and $D = \text{diag}(1, 100)$,

$$H = RDR^T = \begin{pmatrix} 101/2 & -99/2 \\ -99/2 & 101/2 \end{pmatrix}, \quad Hx = \begin{pmatrix} 101/2 \\ -99/2 \end{pmatrix} \approx \begin{pmatrix} 50.5 \\ -49.5 \end{pmatrix}.$$

Newton direction $\propto -H^{-1}Hx = -x = (-1, 0)$.

Adam-style diagonal step (with $P = \text{diag}(1, 0.01)$) is

$$-PHx = -\begin{pmatrix} 1 & 0 \\ 0 & 0.01 \end{pmatrix} \begin{pmatrix} 50.5 \\ -49.5 \end{pmatrix} = -\begin{pmatrix} 50.5 \\ -0.495 \end{pmatrix} \propto (-1, 0.00980198),$$

which matches your $(-1, 0.0098)$.

Geometric intuition (brief).

The true Newton step undoes both the rotation and the spectral gap, so it points exactly opposite to x . A diagonal preconditioner can rescale the axes but cannot *un-rotate* the gradient; after multiplication by H the gradient lies in a rotated direction (large component on the second eigenvector), and P merely shrinks the second coordinate — it leaves a tiny off-axis component. That small transverse component produces a step that is nearly, but not exactly, aligned with $-x$. Over many iterations this residual causes the optimizer to alternate (zig-zag) across the valley instead of moving straight down the eigen-direction.

One quick test of understanding (one sentence):

Why does that tiny transverse component (the ≈ 0.0098 entry) lead to *slow* convergence in the long run, even though it looks numerically negligible on a single step?

can you go over lr schedules for each optimizer?

Sure — but let's keep this in **study-mode style**: I won't dump a giant table. Instead, I'll walk you through the *structure* of LR schedules **one optimizer at a time**, but I need one quick check first so I don't oversimplify or over-assume.

Quick calibration question (one sentence):

When you think of a learning-rate schedule, do you think mainly in terms of

(a) global scaling of the step size over time (cosine, exponential decay),

or

(b) how the *effective* step size changes because of the optimizer's internal state (Adam's bias correction, Muon's reconditioning),

or both?

Tell me which perspective you want to anchor our discussion around.

b

Great — so we'll center everything around **effective learning-rate dynamics**, meaning:

$$\text{effective step on parameter } w = \underbrace{\eta_t}_{\text{optimizer-induced scaling}} \times \underbrace{\text{external lr}}_{\text{momentum / normalization / geometry}}$$

We'll go optimizer by optimizer, keeping things concise and checking your understanding as we go.

Before diving in, here's a quick orienting fact you can keep in mind:

Different optimizers secretly produce *different LR schedules even with the same external lr*.

This is why two runs with the same $\text{lr}=1\text{e-}3$ can behave nothing alike.

I'll treat each optimizer this way:

- what its internal state does to the lr early vs late,
- what invariances shape this,
- how schedules typically interact with it,
- what geometric effect this produces.

Then I'll give you one comprehension question per optimizer.

1. Gradient Descent (with momentum)

Internal LR effect

GD has *no* normalization.

Momentum adds a smoothing but doesn't change the magnitude with any systematic time-dependent bias.

$$v_t = \beta v_{t-1} + (1 - \beta) g_t, \quad w_{t+1} = w_t - \eta_t v_t.$$

The effective LR is basically just:

$$\eta_t^{\text{eff}} \approx \eta_t.$$

No built-in schedule.

What this means

- You *must* explicitly decay η_t as curvature increases during convergence.
- Cosine, step-decay, or exponential decay all directly control stability.

When an LR schedule matters most

- Near minima (Hessian eigenvalues dominate).
- In poorly scaled parameterizations (GD is not invariant).

Comprehension check

Why does GD need *more* aggressive LR decay in highly ill-conditioned directions than Adam?

2. SignGD

Internal LR effect

SignGD update:

$$w_{t+1} = w_t - \eta_t \operatorname{sign}(g_t).$$

This kills the gradient magnitude.

Effective LR per coordinate is exactly:

$$\eta_t^{\text{eff}} = \eta_t,$$

and does **not** depend on curvature, gradient scale, or time.

Key internal behavior

- Steps don't shrink as you approach a minimum (unless you schedule η_t).
- Because the step size is fixed in each coordinate, you get a kind of "quantized" path.

What this means for scheduling

- You must decay lr explicitly or you **never** converge smoothly (you only jitter).
- The typical schedule is rapid exponential decay or large warmup → big decay.
- Warmup is less helpful than in Adam; decay is more important.

Comprehension check

Why can SignGD cross narrow valleys more easily than GD early on, even with the same external lr schedule?

3. Adam

Internal LR effect (the real heart)

Effective step on coordinate i :

$$\Delta w_{t,i} = -\eta_t \frac{m_{t,i}}{\sqrt{v_{t,i}} + \epsilon}.$$

The main LR-shaping mechanisms:

(a) Bias correction

Early in training:

$$\frac{1}{\sqrt{v_t}} \approx \frac{1}{\sqrt{(1 - \beta_2)t \mathbb{E}[g^2]}}$$

leading to a **very large effective lr early on**, because v_t is small.

That's why Adam usually doesn't need a huge warmup.

(b) Long-term normalization

At steady state:

$$\frac{m_{t,i}}{\sqrt{v_{t,i}}} \approx \frac{\mathbb{E}[g_i]}{\sqrt{\mathbb{E}[g_i^2]}} = \frac{1}{\text{RMS of gradients in that coordinate}}.$$

So Adam produces a *coordinate-wise adaptive LR schedule*:

- Large grad variance → smaller lr.
- Small grad variance → larger lr.

This schedule persists indefinitely.

(c) Interaction with external lr schedule

Because internal lr changes automatically:

- External lr decay is **less critical** than in GD.
- Cosine with long plateau is common.
- Exponential decay works, but slow decay often suffices.
- Warmup is mild, often 0–2000 steps.

Geometric effect

Adam gradually equalizes effective steps across coordinates but cannot resolve rotation.

Comprehension check

If a coordinate has extremely high curvature but extremely low gradient variance, how does Adam's internal LR scaling behave on that coordinate?

4. Muon (matrix-aware optimizers)

Muon normalizes whole weight **matrices**, not coordinates.

Its effective lr is:

$$\Delta W_t = -\eta_t \cdot \underbrace{W_t^{-1/2}}_{\text{matrix preconditioning}} \cdot m_t,$$

or variants along these lines.

(Exact form varies, but all Muon-like methods apply matrix-level normalization using Newton–Schulz iterations.)

Internal LR dynamics

Muon's normalization removes:

- scale sensitivity,
- conditioning sensitivity,
- and rotation of curvature.

So the effective step becomes almost proportional to:

$$\eta_t \times (\text{isotropic update}).$$

Time-dependence inside Muon

- Early on, the Newton–Schulz iterations produce rough curvature estimates → larger, slightly noisy normalization.
- Later, the matrix normalization stabilizes and implicitly damps the effective lr in high-curvature directions.

Meaning:

Muon induces its own strong internal lr decay in poorly conditioned directions.

External schedule needs

- Much milder external decay than Adam.
- Warmup is still beneficial because matrix normalization is unstable in the first few steps.
- Cosine sometimes works but isn't required for stability.
- Many Muon users run almost constant external lr.

Geometric effect

Effective step magnitude approaches something like a “natural gradient”—nearly invariant to reparameterizations and rotations.

Comprehension check

Why does Muon need less external LR decay than Adam when approaching a minimum with highly anisotropic curvature?

I appreciated how many comprehensive checks it provided

Your next step

Pick one optimizer (GD, SignGD, Adam, or Muon) and try answering its comprehension question. I'll check your reasoning and guide you to the next layer of understanding.

GD) even if curvature is high, the learning rate is global for all directions

SignGD) Even if these valleys are ill-conditioned, SignGD's gradient will remain constant rather than being shrunk by the magnitude

Adam) Low gradient variance -> high lr even if extremely high curvature

Muon) muon takes into consideration curvature (decay when poor conditioning) whereas Adam doesn't

how does SignGD relate to MuP

Good answers — you're reasoning at the right level. I'll first **tighten your statements**, then answer your new question: **SignGD vs MuP**.

1. Quick corrections / sharpening (one-liners)

GD

Your idea is right; sharper version:

GD has a *single global step size*, so large curvature in any direction forces the LR down for all directions.

That's exactly why ill-conditioning kills GD.

SignGD

You said:

gradient remains constant rather than being shrunk by magnitude

Better phrasing:

SignGD discards gradient magnitude, so curvature cannot reduce the step size in steep directions — it only affects *sign consistency*, not step length.

This is why it is robust early, but unstable near minima without decay.

Adam

Your statement is correct but here's the geometric version:

Adam uses historical *variance*, not curvature.
High curvature with consistent gradients → **large effective LR**.
Low curvature with noisy gradients → **small effective LR**.

Adam tracks *statistics*, not geometry.

Muon

Good — and here's a stronger version:

Muon rescales directions based on *matrix structure* and approximate spectral geometry — not gradient noise — so it shrinks steps in directions that are geometrically stiff even if gradients look harmless.

Muon tracks **shape of the surface**, Adam tracks **shape of gradient history**.

Now let's answer your actual question.

How does SignGD relate to MuP?

They live in **very different conceptual layers** but they touch the same phenomenon from opposite sides:

Concept	SignGD	MuP
Object of control	Gradient update	Parameterization & initialization
What is normalized	Gradient direction	Weight scale across width
Level	Optimization algorithm	Scaling law / training invariance
Problem addressed	Gradient magnitude instability	Width dependence of training
Invariant to	Gradient scale	Model width
Relationship	Indirect	Structural

They are not variants of each other — but they solve **related failures** in deep learning from totally different ends.

2. What MuP *actually* does (in one paragraph)

MuP (Maximal Update Parametrization) ensures:

As you scale model width, the *function update magnitude* stays constant.

Ordinary parameterizations have this pathology:

- Double width → gradients shrink
- Step size becomes inconsistent
- Training dynamics change with model size

MuP rescales:

- initialization,
- learning rates,
- certain parameter blocks

so that:

SGD behaves predictably at any width.

It makes **GD itself scale-invariant in width**.

3. What SignGD is doing to *gradients* vs what MuP does to *parameters*

SignGD makes this transformation:

$$g \rightarrow \text{sign}(g)$$

It enforces:

All coordinates receive identical magnitude updates regardless of scale.

This makes SignGD invariant to:

- gradient explosion,
 - gradient vanishing,
 - per-layer scale mismatch
-

MuP enforces:

expected update magnitude $\approx$ constant across width

This occurs through:

- scaled initialization,
- scaled learning rates per parameter class,
- scaled residual branches.

This makes SGD invariant to:

- architecture size,

- width-dependent shrinkage,
- training instabilities during scaling.

They meet here:

Phenomenon	Without control	SignGD	MuP
Scale mismatch	Training unstable	Ignored	Solved
Width increase	Gradients shrink	Masked	Eliminated
Effective LR	Implicit chaos	Equal per coordinate	Equal per neuron
Sensitivity	Extreme	Blunted	Calibrated

4. Why SignGD is *not* a replacement for MuP

If you train a wide model poorly scaled:

- SignGD will still push parameters → but geometry stays wrong
- MuP fixes the **model**, not the optimizer

So:

SignGD hides scale problems; MuP removes them.

5. Subtle connection (the deep insight)

They both enforce invariance — but at different layers of abstraction:

- **SignGD** enforces invariant *optimization steps*.
- **MuP** enforces invariant *training dynamics*.

If you use MuP properly, you don't need SignGD for scale issues — because the gradients are already well-scaled.

If you use SignGD, you don't force correct geometry — you just ignore magnitude.

6. Your test question (single, conceptual)

Answer in one sentence:

If MuP already enforces scale-invariant training dynamics, what advantage (if any) would SignGD still provide?

Take a shot — your answer should reveal whether you see the difference between **geometry** and **robustness** clearly.

It still provides quantized optimization steps whereas mup focuses on invariance to width

Good — that's directionally right. Let me sharpen it so you get the *exact* conceptual split.

Your answer (rewritten succinctly):

SignGD gives robustness by quantizing updates
MuP gives width-invariant training dynamics

Accurate, but here's the more precise and deeper version you want to have in your head.

The precise reason SignGD can still matter under MuP

Even when MuP fixes scaling and width-dependence, SignGD still changes the *shape* of the update rule.

MuP does *not* make SGD behave like SignGD.

It only rescales parameters so the **magnitudes** of updates are consistent across width.

So the real answer is:

SignGD provides robustness to poorly conditioned curvature by discarding gradient magnitudes.
MuP does not fix curvature; it only fixes scaling with width.

That's the crisp distinction.

Why this matters

Here's the conceptual stack:

- 1. MuP fixes scale:**
Makes wide models train equivalently to small ones.
- 2. SignGD fixes magnitude sensitivity:**
Removes dependence on gradient *size*, which GD and Adam still depend on.

These two dimensions are orthogonal:

- MuP is about **parameter scaling and width invariance**.
- SignGD is about **direction-only optimization**.

Visualizing the difference

Imagine you have:

- a wide transformer (MuP controls scaling),
- but a very ill-conditioned loss surface (tall, narrow valleys).

Even with MuP:

- GD still makes tiny steps in steep directions.
- Adam still amplifies low-variance gradients even in stiff directions.
- SignGD still takes full unit steps in each coordinate.

MuP does not prevent curvature-induced issues.

SignGD does.

The fully correct takeaway

Here is your final “correct” version of the answer, stated cleanly:

Even under MuP, SignGD preserves its unique advantage: it *completely eliminates magnitude information* from the gradient, giving steps that are robust to curvature and gradient scaling. MuP makes the optimization dynamics width-invariant, but it does not remove curvature sensitivity. Therefore, SignGD still changes the character of the update in ways MuP does not address.

This is the clearest way to articulate it in interviews or research writing.

If you want next, I can lay out **a full invariance table** comparing:

- GD
- SignGD
- Adam
- Muon
- MuP (parameterization, not optimizer)

across:

- gradient-scale invariance
- parameter-scale invariance
- width invariance
- curvature sensitivity
- induced norm
- effective geometry

Just tell me if you want it.

yes

Let’s build the table, but in **study-mode** we’ll do it in two short steps:

1. I’ll give you the *skeleton* of the table (the categories).
2. You fill in **one row**, and I’ll correct/complete it; then we continue.

That way you actually *learn the invariances* rather than just reading them.

Here are the categories we’ll use:

Invariance / Sensitivity dimensions

1. Gradient-scale invariance
2. Parameter-scale invariance
3. Width invariance
4. Curvature sensitivity
5. Induced geometry / preconditioning
6. Sensitivity to reparameterization (e.g., linear layer scaling $W \rightarrow cW$, $x \rightarrow x/c$)

Now the table skeleton:

Optimizer / Parametrization	Grad-scale inv.	Param-scale inv.	Width-inv.	Curvature sensitivity	Geometry type
--------------------------------	--------------------	---------------------	------------	--------------------------	---------------

Before I fill it:

Can you pick ONE of the four: GD, SignGD, Adam, Muon — and fill *only the first two columns* (“Grad-scale invariance” and “Param-scale invariance”)?

Choose whichever one you’re most confident in. I’ll check and then continue building the table with you.

GD: no, no
SignGD: yes, no
Adam: yes, no
Muon: yes, no

Good — gradient-scale invariance is right for all except GD.
Parameter-scale invariance is trickier: all four entries are actually **no**, but the *reason* differs between them.

Let’s lock in the partial table, then move carefully to the next columns.

Here’s what we have so far:

Optimizer	Grad-scale inv.?	Param-scale inv.?
GD	no	no
SignGD	yes	no
Adam	yes	no
Muon	yes	no

Everything so far is correct.

Before we continue, let’s check understanding with one quick diagnostic question.

Why is Adam *not* parameter-scale invariant, even though it normalizes the gradient by its second moment?

Give me a short reasoning — one or two sentences. I’ll use that to place the next columns.

because the dimension still scales the magnitude of the mean and second moment

Good — that's the right intuition. Let me tighten it slightly so you see the exact mechanism:

- When you scale a parameter $w \rightarrow cw$, the gradient also scales $g \rightarrow cg$.
- In Adam, both first and second moments scale, but **second moment scales quadratically**:
 $m \rightarrow cm, v \rightarrow c^2v$.
- So the ratio $m/\sqrt{v}$ becomes
 $\frac{cm}{\sqrt{c^2v}} = \frac{cm}{c\sqrt{v}} = m/\sqrt{v}$,
but the **global step size and weight decay terms do not transform compatibly**, so the *full* update is not invariant.

That's the key idea you were gesturing at.

Now we can extend the table to the next column.

Add Column 3: Width Invariance

This means: does the optimizer produce *equivalent dynamics* when model width changes (e.g., number of channels, hidden units)?

Only **MuP (parameterization)** gives true width invariance. None of the optimizers do this alone.

So the next column:

Optimizer	Grad-scale inv.	Param-scale inv.	Width-inv.?
GD	no	no	no
SignGD	yes	no	no
Adam	yes	no	no
Muon	yes	no	no
MuP (parametrization)	—	—	yes

Notice: MuP is not an optimizer. It's a **parameterization**.
Optimizers inherit invariances *from* MuP but don't get width invariance on their own.

Before we continue to curvature sensitivity, I want to quickly check that this is clear:

One-sentence check:
Why does MuP — and not Adam/Muon/SignGD — give width invariance?

Your answer will determine how I fill out the curvature column.

because it scales the learning rate by 1/ width of the layer

Thought for 4 seconds

Nice — that's the right intuition. One small clarification: **MuP** doesn't literally always scale by $\frac{1}{\text{width}}$ for every parameter; it *chooses initialization and per-parameter learning-rate scalings so the expected update magnitude stays constant as width changes*. Practically that often looks like dividing by a width-dependent factor (different blocks use different powers), but the key effect is **making updates width-invariant**, not enforcing a single universal $1/\text{width}$ rule.

Below I've completed the full invariance table using short, precise entries. Read each line, then pick one optimizer you want a short derivation for (one-step math) to check your understanding.

Summary Table that it asked me to make interactively was very helpful

Optimizer / Parametrization	Grad-scale inv.	Param-scale inv.	Width-inv.?	Curvature sensitivity	Induced geometry / preconditioning	Sensitivity to cW, n
GD / SGD (+ momentum)	no	no	no	high (very sensitive) — slow on ill-conditioned directions	Euclidean / ℓ_2 steepest-descent (no preconditioning)	high — gradient
SignGD / signSGD	yes	no	no	low → moderate (magnitude discarded; direction-only can ignore curvature magnitude)	ℓ_∞ -style step (coordinate-wise sign) — extreme quantization	high — change steps
Adam / AdamW	yes	no	no	moderate — compensates per-coordinate scale, but misses cross-coordinate curvature	Diagonal adaptive preconditioner (RMS-based) — approx $\text{diag}(1/\sqrt{v})$	moderate — normal couple decay
Muon (matrix-aware)	yes	no	no	low — explicitly reduces spectral conditioning across rows/cols	Matrix-level spectral preconditioning / orthogonalization (approximate whitening)	lower — certain because covarian
MuP (parametrization)	— (n/a)	— (n/a)	yes	n/a (it's a parametrization) — it stabilizes gradient magnitudes across width	Changes the effective geometry by rescaling init/LR per param class (not a preconditioner)	design — scaling rescaled neutral

Notes (one-line each):

- *Grad-scale inv.*: whether multiplicative scaling of gradients (global or per-coordinate positive scalar) leaves the update unchanged.
- *Param-scale inv.*: whether rescaling parameters (e.g., $w \mapsto cw$) leaves the optimizer update law invariant (most are **not**).
- *Curvature sensitivity*: how much the optimizer's raw step responds to Hessian eigenvalue spread — high sensitivity means small eigenvalues slow progress.

- *Induced geometry*: the norm/metric under which the optimizer is (locally) steepest-descent-like.
- *Renormalization sensitivity*: how badly the optimizer reacts to equivalent-function